Отчёт по лабораторной работе
«Отношения в базе данных и их тестирование (Django)»

09.11.2026

1. Цель работы и постановка задачи

Цель работы — разработать на Django небольшой стартап «Конные туры» с отношениями в базе данных (один к одному, один ко многим, многие ко многим) и проверить бизнес-правила модульными тестами. Итог: приложение tours и 54 автоматических тестов, все проходят.

По заданию нужно было проверить, что: один человек может купить только один тур на определённый день и время; один человек может оставить несколько комментариев, но только к тем турам, в которых он был; у тура есть категории, например «конный», «пеший» или «на мото».

Отдельным пунктом требовалось протестировать все отправки данных (POST-формы) и сделать документацию через django docs. Средства разработки: Python 3.13, Django 6, SQLite, django.contrib.admindocs.

2. Описание проекта

Проект называется config, вся логика — в отдельном приложении tours, как мы делали в прошлых работах. В нём шесть моделей:

Модель	**За что отвечает**
Profile	Профиль пользователя (телефон, дата рождения). Создаётся автоматически сигналом при регист
Category	Категория тура: «конный», «пеший», «на мото»
Tour	Тур: название, описание, цена, длительность
TourSession	Слот тура на конкретный день и время
Booking	Покупка тура пользователем
Comment	Комментарий и оценка (1–5) к туру

Связи между моделями:

Тип связи	**Где реализована**
Один к одному (1:1)	User — Profile (OneToOneField)
Один ко многим (1:N)	Tour — TourSession, User — Comment, User — Booking, TourSession — Booking
Многие ко многим (M:N)	Tour — Category (ManyToManyField)

Ключевые решения: ограничение «один тур на день и время» проверяется и в методе clean() модели Booking, и в BookingForm, чтобы ошибку было видно и в админке, и на сайте; плюс UniqueConstraint на пару «пользователь + слот». Комментарий можно оставить, только если есть бронирование этого тура (clean() в Comment и CommentForm). Профиль создаётся сигналом post_save. Документация подключена через django.contrib.admindocs и открывается по адресу /admin/doc/.

3. Тестирование

Тесты написаны в файле tours/tests.py: 54 тестов в 13 классах django.test.TestCase. Каждый класс готовит свои данные в setUp или функциях-хелперах, поэтому тесты независимы.

Класс тестов	**Тестов**	**Что проверяется**
BookingRuleTests	7	Правило «1 человек = 1 тур на день и время»
BookingViewPostTests	4	POST покупки: успех, отказ, аноним
CommentRuleTests	5	Комментарии только к посещённым турам, оценка 1–5
CommentViewPostTests	5	POST комментария: успех, отказ, несколько подряд
ManyToManyRelationTests	3	Связь M:N — категории тура видны с обеих сторон
OneToManyRelationTests	5	Связь 1:N — у тура много слотов, у пользователя много комментариев, кас
OneToOneRelationTests	4	Связь 1:1 — профиль создаётся автоматически, поля сохраняются, удаляетс
TourCreateViewTests	3	POST создания тура
TourDeleteViewTests	3	POST удаления тура
TourFormTests	3	Форма тура: валидные и невалидные данные
TourReadViewTests	7	Список, поиск, фильтр по категории, 404
TourUpdateViewTests	4	POST редактирования тура
UrlTests	1	Все URL ведут на нужные представления
Итого	54	

Пример теста на главное правило — запрет второго тура на то же время:

```
def test_cannot_buy_two_tours_same_datetime(self):
    """Нельзя купить 2 тура на один и тот же день и время (clean)"""
    make_booking(self.user, self.s1)
    b = Booking(user=self.user, session=self.s2)
    with self.assertRaises(ValidationError):
        b.full_clean()
```

4. Результаты

Все 54 тестов пройдены за 5.301 с (Ran 54 tests ... OK), ошибок и падений нет. Тесты запускаются командой python manage.py test.

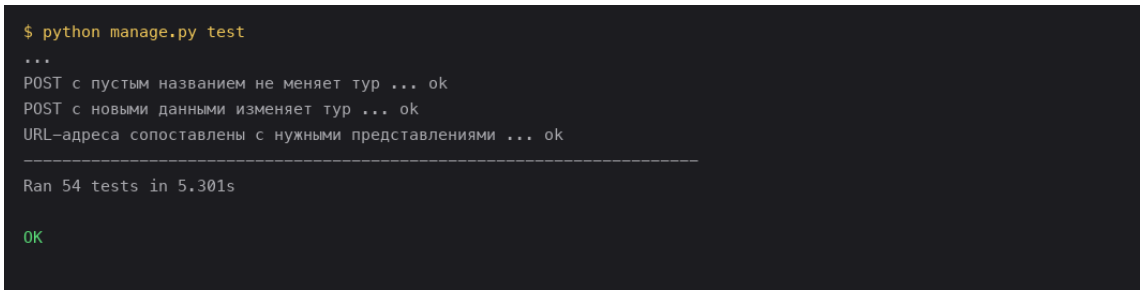


Рисунок 1 — запуск тестов

5. Вывод

Собран стартап «Конные туры» со всеми тремя типами связей, и все правила из задания подтверждены тестами. Из полезного: оказалось, что бизнес-правило лучше проверять и в модели (clean), и в форме — иначе через админку можно было бы обойти ограничение. Ещё удобно вышло с сигналом post_save для профиля: руками создавать его не нужно нигде, даже в тестах. Документация через admindocs поднялась сразу, потому что у всех

моделей и представлений были docstring'и.

Исходный код, тесты и этот отчёт лежат в репозитории github.com/Erbo102804/Lab-work-7-.